Milestone 1 – Task 1: Research Report

AI Code Review & Security Analysis Agent · Infosys Springboard Internship · Week 1–2

1. Purpose of this Report

This report documents the research required before designing and building the platform. It covers four areas that map directly to what the system will do:

Research area	Which part of the platform it feeds
OWASP vulnerability standards	Security Vulnerability Agent, severity scoring
Secure coding guidelines	Remediation Agent, knowledge base content
Code smell patterns	Code Analysis Agent
RAG architecture	Knowledge base, Conversational Code Assistant, grounding of Remediation Agent

Core idea of the project in one line: a developer submits Python/Java code, a pipeline of specialised AI agents finds quality and security problems, proposes fixes grounded in trusted security standards, and a chat assistant explains anything the developer does not understand.

---

2. OWASP Vulnerability Standards

2.1 What OWASP is

The Open Worldwide Application Security Project (OWASP) is a non-profit foundation that publishes free, community-reviewed material on application security. It is the de facto reference used by auditors, companies and tools. The relevant OWASP resources for this project are:

- **OWASP Top 10**: awareness list of the ten most critical web application security risks, updated every few years.
- **OWASP ASVS (Application Security Verification Standard)**: a detailed checklist of security requirements, organised by level (L1 to L3).

- **OWASP Cheat Sheet Series**: short, practical, topic-specific guides (SQL Injection Prevention, XSS Prevention, Password Storage, Deserialization, etc.). This is the **best source for the knowledge base** because each sheet is concise, actionable and has code examples.
- **OWASP Code Review Guide / Testing Guide**: how to review code and test for flaws.

2.2 OWASP Top 10 (2021 edition – the baseline most tools and courses use)

ID	Category	What it means	Typical code-level signal
A01	Broken Access Control	Users can act outside their permissions	Missing authorization checks, IDOR (`/orders/{id}` with no owner check), path traversal
A02	Cryptographic Failures	Sensitive data exposed due to weak/absent crypto	MD5/SHA1 for passwords, hardcoded keys, HTTP instead of HTTPS, ECB mode
A03	Injection	Untrusted input is interpreted as a command or query	String-concatenated SQL, `os.system(user_input)`, `eval()`, XSS output without encoding
A04	Insecure Design	Missing security controls at design level	No rate limiting, no input validation strategy
A05	Security Misconfiguration	Unsafe defaults or settings	`DEBUG=True`, default credentials, verbose error pages, permissive CORS
A06	Vulnerable and Outdated Components	Using libraries with known CVEs	Old pinned dependency versions
A07	Identification and Authentication Failures	Weak login/session handling	Plain-text passwords, no lockout, weak session tokens
A08	Software and Data Integrity Failures	Trusting unverified code or data	Insecure deserialization (`pickle.loads`, `ObjectInputStream`), unsigned updates

ID	Category	What it means	Typical code-level signal
A09	Security Logging and Monitoring Failures	Attacks go unnoticed	No logging of auth failures; logging passwords
A10	Server-Side Request Forgery (SSRF)	Server fetches attacker-controlled URLs	`requests.get(user_url)` with no allow-list

> **Note on versions:** OWASP has published a newer Top 10 edition (2025) that reorders and renames some categories (for example, adds supply-chain and exceptional-condition handling topics). Confirm the current list at owasp.org/Top10 before finalising. A good design choice is to **store the OWASP category as a configurable field**, so moving between editions needs no code change.

2.3 Vulnerabilities the project must detect (from the project brief)

Vulnerability	OWASP category	CWE ID	Vulnerable pattern (Python)	Secure pattern
SQL Injection	A03	CWE-89	`cur.execute("SELECT * FROM users WHERE id=" + uid)`	`cur.execute("SELECT * FROM users WHERE id=%s", (uid,))`
Cross-Site Scripting (XSS)	A03	CWE-79	`return "<h1>" + name + "</h1>"`	Auto-escaping templates / `html.escape(name)`
Hardcoded secrets	A02 / A07	CWE-798	`API_KEY = "sk-12345..."`	`os.environ["API_KEY"]` or a secrets manager
Broken access control	A01	CWE-284 / CWE-639	Returning a record by ID without checking owner	Verify `record.owner_id == current_user.id`
CSRF	A01 / A04	CWE-352	State-changing POST with no token	CSRF tokens, SameSite cookies
Insecure authentication	A07	CWE-256 / CWE-327	`hashlib.md5(password)`	`bcrypt` / `argon2` with salt
Command injection	A03	CWE-78	`os.system("ping " + host)`	`subprocess.run(["ping", host])` without shell

Vulnerability	OWASP category	CWE ID	Vulnerable pattern (Python)	Secure pattern
Insecure deserialization	A08	CWE-502	`pickle.loads(data)`	JSON or signed data; avoid pickle on untrusted input
Path traversal	A01	CWE-22	`open("/files/" + filename)`	Normalise path, check it stays inside base dir

Java equivalents to support: `Statement.executeQuery("..." + input)` vs `PreparedStatement`; `Runtime.exec(userInput)`; `new ObjectInputStream(untrusted)`; `MessageDigest.getInstance("MD5")`; unescaped `response.getWriter().println(param)`; hardcoded `password = "admin123"`; XML parsers without disabling external entities (XXE, CWE-611).

2.4 Related standards you should know

- **CWE (Common Weakness Enumeration):** numeric catalogue of weakness types. Using CWE IDs makes every finding standard and traceable.
- **CVE:** specific vulnerabilities in specific products (not for our own code, but for dependency issues).
- **CVSS (Common Vulnerability Scoring System):** 0–10 numeric severity (None, Low 0.1–3.9, Medium 4.0–6.9, High 7.0–8.9, Critical 9.0–10.0). We will use a simplified CVSS-inspired scheme (see the Design document).
- **SANS/CWE Top 25:** list of most dangerous software weaknesses, useful for prioritisation.

---

3. Secure Coding Guidelines

Secure coding guidelines are the "how to do it right" counterpart to vulnerability lists. The remediation agent and the chat assistant will both draw on them.

3.1 Core principles (apply to every language)

1. **Validate input, encode output.** Treat all external data (forms, headers, files, APIs) as untrusted. Validate with allow-lists; encode for the target context (HTML, SQL, shell).
2. **Use parameterised interfaces.** Prepared statements, ORM query builders, argument-list process calls.
3. **Least privilege.** Code, DB accounts and users get only the permissions they need.

4. **Defence in depth.** Never rely on a single control.
5. **Fail securely.** Errors must deny access, and must not leak stack traces or internals.
6. **Secure by default.** Safe configuration out of the box.
7. **Never hardcode secrets.** Use environment variables or a vault; rotate if leaked.
8. **Use proven cryptography.** No custom crypto; use bcrypt/argon2 for passwords, AES-GCM for encryption, TLS in transit.
9. **Keep dependencies updated** and scan them.
10. **Log security events, but never log secrets.**

3.2 Language-specific guidance

Python: avoid `eval`, `exec`, `pickle` on untrusted data, `yaml.load` (use `yaml.safe_load`), `subprocess(..., shell=True)`; use `secrets` module not `random` for tokens; use `defusedxml` for XML; do not use `assert` for security checks (removed with `-O`); use `with` for resource handling.

Java: use `PreparedStatement`; avoid native serialization of untrusted data; use `SecureRandom`; disable XXE on XML factories; use OWASP Java Encoder for output; avoid catching and swallowing generic `Exception`; close resources via try-with-resources.

3.3 Sources to index in the knowledge base

Source	Why it is valuable
OWASP Cheat Sheet Series	Short, actionable, with code examples (best RAG material)
OWASP Top 10 pages	Category definitions, prevention advice, example scenarios
OWASP ASVS	Verifiable requirements to cite in findings
CWE entries for the top weaknesses	Standard definitions and mitigations
SEI CERT Secure Coding Standards (Java, C)	Rule-level guidance with compliant/non-compliant examples
Official language docs (Python `subprocess`, `hashlib`, `secrets`; Java `PreparedStatement`)	Authoritative API behaviour

Source	Why it is valuable
Project-authored fix notes	Short curated "vulnerable → fixed" examples written by you

All of these are openly licensed or freely readable. Store the source URL and licence note as chunk metadata so answers can cite them.

---

4. Code Smell Patterns

4.1 Definition

A **code smell** is a surface symptom in code that *may* indicate a deeper design problem. It is not a bug and the code still works, but it makes code harder to read, change and test. The catalogue comes mainly from Martin Fowler's *Refactoring* and Robert Martin's *Clean Code*.

4.2 Smell catalogue (grouped)

Group	Smell	Description	Typical fix
Bloaters	Long Method	Function does too much (e.g., >40 lines)	Extract Method
	Large Class (God Class)	Class with too many responsibilities	Extract Class
	Long Parameter List	More than ~4–5 parameters	Parameter Object
	Primitive Obsession	Primitives instead of small value objects	Replace with class
Object-orientation abusers	Switch/long if-else chains	Type-checking conditionals	Polymorphism
	Refused Bequest	Subclass ignores parent behaviour	Rework hierarchy

Group	Smell	Description	Typical fix
Change preventers	Divergent Change / Shotgun Surgery	One change touches many places, or one class changes for many reasons	Move Method, Extract Class
Dispensables	Duplicate Code	Same logic copied	Extract Method / shared module
	Dead Code	Unused variables, functions, branches	Delete
	Comments as deodorant	Comments hiding unclear code	Rename, restructure
Couplers	Feature Envy	Method uses another class's data more than its own	Move Method
	Message Chains	`a.b().c().d()`	Hide Delegate
Other	Magic Numbers / Strings	Unexplained literals	Named constants
	Deep Nesting	More than 3 levels of indentation	Guard clauses, extract
	Poor Naming	`x`, `data2`, `tmp`	Rename
	Empty/Broad Exception Handling	`except: pass`, `catch (Exception e) {}`	Catch specific, log, handle

4.3 Measurable metrics (the Code Analysis Agent's quantitative layer)

Metric	Meaning	Common threshold
Cyclomatic complexity	Count of independent paths through a function	>10 moderate, >20 high
Cognitive complexity	How hard code is to understand (penalises nesting)	>15 flag
Function length (LOC)	Lines per function	>40–50 flag
Nesting depth	Max block depth	>3 flag

Metric	Meaning	Common threshold
Parameter count	Params per function	>5 flag
Class size / method count	Size of class	>20 methods flag
Duplication	Repeated blocks	>5% or repeated 6+ lines
Maintainability Index	Composite score (0–100)	<20 poor

4.4 How smells are detected

- **Static, rule-based:** parse code into an **AST** (Abstract Syntax Tree) and compute metrics. Python: built-in `ast` module, plus `radon` (complexity), `pylint`, `flake8`. Java: `javalang` or `tree-sitter`, tools like PMD, Checkstyle, SpotBugs.
- **LLM-based:** an LLM reads code and reasons about design issues that rules cannot express (poor abstraction, misleading names, SRP violations).
- **Hybrid (recommended):** deterministic metrics give precise, repeatable facts; the LLM adds explanation and judgment. The metrics are passed *into* the LLM prompt, which reduces hallucination and gives line-accurate output.

4.5 Existing tools to benchmark against

Tool	Language	Focus
Bandit	Python	Security issues (rule-based)
Semgrep	Multi	Pattern-based security and quality rules
SonarQube	Multi	Quality, smells, security hotspots
PMD / SpotBugs / Checkstyle	Java	Smells, bugs, style
Pylint / Radon	Python	Quality and complexity

These give you a ground truth to compare your agents against and are excellent test oracles in Milestone 4.

---

5. RAG Architecture

5.1 Why RAG

A plain LLM can hallucinate, has a knowledge cutoff, and cannot cite sources. **Retrieval-Augmented Generation (RAG)** fixes this by retrieving relevant passages from a trusted knowledge base and giving them to the LLM as context. For security advice this is essential: a fix recommendation must be traceable to an authoritative source such as an OWASP cheat sheet.

5.2 The two phases

Phase A – Indexing (offline, done once and refreshed when documents change)

```
Documents (OWASP, CERT, CWE...)
  → Load & clean
  → Chunk (split into passages)
  → Embed (text → vector)
  → Store in vector database (with metadata)
```

Phase B – Query (online, per question or per finding)

```
User question / finding description
  → (optional) query rewriting
  → Embed query
  → Similarity search in vector DB (top-k)
  → (optional) rerank / filter by metadata
  → Build prompt: instructions + retrieved context + question
  → LLM generates answer with citations
```

5.3 Component decisions

Chunking. Chunk size is a trade-off: too small loses context, too large dilutes relevance.

Strategy	Description	Verdict for this project
Fixed-size	Every N tokens	Simple, but cuts mid-concept
Recursive character	Split on paragraphs, then sentences	Good default

Strategy	Description	Verdict for this project
Structure-aware	Split on headings/sections	**Best for cheat sheets and standards**
Semantic	Split where meaning shifts	Higher cost, optional

Recommendation: structure-aware splitting by heading, with a recursive splitter inside long sections. Target **400–800 tokens per chunk with 10–15% overlap**. Keep code examples intact in one chunk. Prefix each chunk with its heading path (e.g., "SQL Injection Prevention > Defense Option 1: Prepared Statements").

Embeddings. Convert text into vectors so semantic similarity can be measured by cosine similarity.

Model	Notes
`all-MiniLM-L6-v2` (sentence-transformers)	Free, local, fast, 384 dimensions, good for a prototype
`BAAI/bge-small-en-v1.5` / `bge-base`	Free, stronger retrieval quality
OpenAI `text-embedding-3-small`	Hosted, paid, high quality

Recommendation: start with a free local model so the project has no cost or API dependency. The same model must be used for indexing and querying.

Vector store.

Store	Notes
ChromaDB	Easy, persistent, metadata filtering, beginner friendly (**recommended**)
FAISS	Very fast library, no built-in metadata or persistence layer
Qdrant / Weaviate / pgvector	Production-grade, heavier to set up

Retrieval. Top-k similarity (k = 4–6) is the baseline. Improvements: **MMR** for diversity; **metadata filters** (language = python, owasp_category = A03); **hybrid search** combining keyword (BM25) and vector; **re-ranking** with a cross-encoder.

Generation. The prompt must instruct the LLM to answer *only* from supplied context, to cite source IDs, and to say "not found in knowledge base" if context is insufficient. Lower temperature (0–0.3) improves factuality.

5.4 How RAG is used in this platform

Use	Query	Result
Remediation Agent	"Fix SQL injection in Python cursor.execute" (built from the finding)	Prevention guidance and code example from cheat sheets
Security Agent (optional)	Finding description	Confirm category and CWE
Conversational Assistant	Developer's question plus current finding context	Grounded explanation with citations

5.5 Evaluating RAG quality

- **Retrieval:** Recall@k and MRR on a hand-made set of 20–30 question → expected-source pairs.
- **Generation:** *faithfulness* (is the answer supported by context?), *answer relevance*, *citation correctness*. Frameworks such as RAGAS can automate this.
- **Common failure modes:** chunks too small, wrong embedding model for domain, missing metadata filtering, retrieval returns near-duplicates, LLM ignores context. Each has a matching fix above.

5.6 Multi-agent concepts (supporting research)

An **agent** is an LLM (plus tools and instructions) with a defined role. A **multi-agent system** splits a complex job into specialised agents coordinated by an **orchestrator**. Benefits: focused prompts, easier testing, parallel execution, clear responsibility. Patterns relevant here: **pipeline (sequential)**, **fan-out/fan-in (parallel then merge)**, **shared state** passed between agents. Frameworks to know: LangGraph, CrewAI, AutoGen, or a simple custom orchestrator. Details are in the Design document.

---

6. Key Design Conclusions Drawn from the Research

1. Use a **hybrid detection approach**: AST/static rules for facts and metrics, LLM for reasoning and explanation.
2. Tag every finding with **OWASP category + CWE ID + severity + line numbers**.
3. Build the knowledge base from **OWASP Cheat Sheets first**, then Top 10, ASVS, CERT and CWE entries.
4. Use **structure-aware chunking**, a **free local embedding model**, and **ChromaDB** with metadata filters.
5. Make the Remediation Agent and chat assistant **retrieval-grounded with citations**.
6. Keep the standard version (OWASP 2021 vs 2025) **configurable data**, not hardcoded.

7. Self-Check Questions (likely mentor/viva questions)

1. What is the difference between OWASP Top 10, ASVS and the Cheat Sheets?
2. Why does string-concatenated SQL cause injection and how does parameterisation stop it?
3. What is the difference between a bug, a vulnerability and a code smell?
4. What does cyclomatic complexity measure and what is a typical threshold?
5. Why combine static analysis with an LLM instead of using only one?
6. Why does chunk size affect retrieval quality?
7. What is the role of the embedding model and why must indexing and query use the same one?
8. How does RAG reduce hallucination?
9. What would you do if retrieval returns irrelevant chunks?

8. References

- OWASP Top 10 – owasp.org/Top10
- OWASP Cheat Sheet Series – cheatsheetseries.owasp.org
- OWASP ASVS – owasp.org/www-project-application-security-verification-standard
- CWE – cwe.mitre.org
- FIRST CVSS – first.org/cvss
- SEI CERT Coding Standards – wiki.sei.cmu.edu/confluence/display/seccode

- Refactoring Guru, Code Smells – refactoring.guru/refactoring/smells
- Fowler, M., *Refactoring: Improving the Design of Existing Code*
- Lewis et al. (2020), *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*
- ChromaDB docs – docs.trychroma.com; sentence-transformers docs – sbert.net